Contents

(Steps for accomplishments) 2
[HOW to steps] 3
Step1 : Detection 3
Step 2: 3
Step 3 : Containment 4
[Resolving the issue] 5
Disable a Bad Deployment 5
[What is a bad deployment] 5
Scale up servers 6
[Cache] 7
[Whiteout Cache] 7
[With Cache enabled] 7
[Cache Overload] 7
[Symptoms] 7
[Real Life Example] 8
[Actions to take] 8
[Application Overload] 8
[Affected Components] 8
[Symptoms] 8
[result of an application overload] 9
[Questions to account for] 9
[Actions to take] 9
[Tools] 9
[Containment] 10
[Without Containment] 10
[With containment] 10

(Steps for accomplishments)

During an incident

1.Detecing the problem

2.Accessing Impact

What is affected?

All users?

The entire application down?

What functionality is broken?

What is the blast radius?

What is the business impact?

What severity level is it?

3. restoring service

Prioritizing speed and getting the user-facing functionality back online (“Stopping the bleeding”) rather than spending time hunting for root cause.

4. Communicating updates

5. Finding root cause

6.Preventing future occurrences.

[HOW to steps]

Step1 : Detection – an incident starts when something abnormal is detected:

EX: Website goes down, API latency spikes, Database connections fail, Error rate increase, Customer report issues.

tools:

- Grafana
- Datadog
- Azure monito
- Splunk
- Dynatrace
- PagerDuty

[Questions to ask]

What failed?

When did it start?

Is it still happening?

How many users are affected?

Step 2:

Triage – Determine severity

[Severity Levels]

SEV-1 :

Entire production system down

Major business impact

SEV-2 **:**

Partial outage

Large user impact

SEV-3

Minor degradation

[When determining the level it is important to notice] :

Is production affected?

Is revenue affected ?
is customer data at risk ?

Can users still perform critical functions?

Step 3 : Containment

Stop the solution from getting worse.

[Example]

- Disable a bad deployment
- Scale up servers
- Block malicious traffic
- Fail over to back up systems

[WithoutContaoment]

More customers are affected

More systems fail

Data could be lost

Revenue loss increase

Recovery becomes hard

NOTE: A good SRE tries to reduce the “blast radius” of an incident.

[Blast Radius] : How much of the environment is affected?
EX: one server fails = small blast radius

[Resolving the issue]

Disable a Bad Deployment

[What is a bad deployment] - when a dev releases a new version of software and into production.

EX:

Version 1.0 -> working

Version 1.1 -> not working (has a bug)

This can cause :

Increase login failure increase

Error rate jump

Customers cannot access services.

This is relatable to incidents because SRE would ask :

What changed?

- New application releases
- Configuration updates
- Security changes
- Infrastructure modifications.

[real life example]

1) Casino transactions start failing
2) Casinos cannot process deposits
3) Customer complain

[Actions]

- Start by a rollback of the release
- Restore payment processing
- Investigate later

This is so revenue is protected and customers can continue to transaction.

Scale up servers

[What is scaling]

Scaling is Adding resources ;

CPU

Memory

Servers

Kubernetes pods

[Example]

Application designed for **100 users per hour**, but suddenly receives **10,000 users per hour**. Application designed for slow traffic flow

[Common causes:]

- Holiday traffic
- Marketing campaigns
- Casino promotions

NOTE: Thre are several ways to resolve / pass by this type of concern.

[Cache]

[what is cache]

Temporary storage layer that keeps frequently requested data in memory, so applications do not need to repeatedly query slower systems such as databases.

[Example]

User → Database → Response

The application can perform:

User → Cache → Response

[Whiteout Cache]

Customer requests are specifically directed to the client

[With Cache enabled]

If a promotion was active, it would be stored on the cache leaving less request for the database and indirect coverage.

[Cache Overload]

The traffic flow increases passing the cache limitation , and enables cache overload leading traffic flow.

[Symptoms]

Increase latency

Slow application Reponses

Timeouts

Cache misses increase

Data traffic spikes

[Real Life Example]

Large number of customer wanting to purchase a product from a website.

Large request start to increase.

Deposits and withdraws start declining

[Actions to take]

SRE Actions:

Scale cache resources

Add additional cache nodes

Increase memory allocation

Monitor cache hit rate

[Application Overload]

An application receiving more traffic than expected becomes a application overload.

[Affected Components]

CPU

Memory

Threads

Network bandwidth

Available connections

[Symptoms]

High API latency

Login failures

Timeout errors

Service crashes

Increased error rates

Slow customer experience

[result of an application overload]

Assume there are normally 100 users and now there are 1000 users, Transactions start to get slow, fail, and timeouts start occurring.

[Questions to account for]

Is CPU utilization high?

Is memory exhausted?

Is traffic higher than normal?

Are database connections maxed out?

Did a promotion or event increase traffic?

Did a recent deployment introduce inefficiencies?

[Actions to take]

As an SRE

- Scale up servers
- Increase Kubernetes pod count\
- Add additional application instances
- Route traffic to healthy systems
- Enable auto scaling

Keep in mind it is very important to keep production live and run. Then fixing comes after.

[Tools]

Grafana

Azure monitor

[Expectations]

CPU at 95%

Memory exhaustion

Increase API latency

Cache hit rate dropping

[Triage]

To determine the severity we ask
is the overload affected all users?

Is production impacted?

Is revenue impacted?

Can customers still make deposits and withdraws? This classifies for a SEV level.

[Containment]

Scale application servers

Scale cache servers

Block unnecessary traffic

Roll back bad deployment

Fail over back up systems

- **[Goal]** the goal of a containment is so the bug does not spread and cause more internal issues for the product. Stopping the issue from getting worse.
  END GOAL :
  Reducing customer impact
- Limiting the blast radius
- Protecting business operations
- Preventing additional system failures
- Restoring service quickly

[Without Containment]

- More customers are affected
- Revenue loss increases
- Additionally system fail
- Data loss becomes more likely

[With containment]

- Impact is minimized
- Service is restored faster
- The blast radius stays small